

Learnings: Building an LLM from Scratch using Rust

Introduction

These notes capture the primary insights and structural concepts learned while studying the implementation of a language model from scratch using the Rust programming language [?, ?].

1 Part 1: The Tokenizer

The tokenization phase establishes the groundwork for the language model by addressing three core dimensions:

1. Converting text to numbers using Byte Pair Encoding (BPE).
2. Understanding how this step directly shapes everything the model can and cannot do.
3. Analyzing how different vocabulary sizes affect compression, processing speed, and training behavior.

1.1 Core Concepts & Definitions

1. **Tokenizer:** A specialized tool used to convert human-readable text into numerical tokens.
2. **BPE:** The specific subword algorithm used during the tokenization process.
3. **Feste:** The name of the GPT-2 style language model being constructed.
4. **Transformer:** The foundational deep learning architecture underpinning the language model.
5. **Shakespeare:** The dataset source. The model is actively trained on the complete works of William Shakespeare.

1.2 Learning about the Transformer Architecture

A transformer is a deep learning architecture integrated into a language model. It relies on an encoding-decoding structure and is driven by the *Attention Mechanism*.

1.2.1 What is the Attention Mechanism?

This method determines the mathematical relationships between multiple words within a sentence, determining how they correlate with one another.

Unlike conventional neural networks (such as RNNs or CNNs) which process data sequentially—incurring substantial time and storage penalties—the transformer model processes massive strings of data points in parallel. This parallelism accelerates operations like relevance mapping, encoding, and decoding.

1.2.2 Key Features of the Model

- **Long-Range Context:** The model can maintain contextual relevance across long sequences.
- **Parallel Feature Processing:** Parameters like grammar, emotional tone, and syntax are processed concurrently.
- **Multi-Head Self-Attention:** Designed for uncovering and maintaining semantic relationships between disparate words.

- **Positional Encoding:** Injects structural ordering metadata into word vectors to preserve word sequences.

Example: In the sentence “*Ramu goes to temple in his village,*” the word “*his*” refers to the name “*Ramu*”. The model identifies this connection using its self-attention layer.

1.2.3 Applications & Benefits

Transformers excel in specific real-world tasks such as training customer support chatbots and generating condensed medical report summaries. Their primary architectural benefits are long-range context handling, execution concurrency, and scalability.

1.3 What Are We Building?

The model, **Feste**, utilizes a GPT-2 style architecture built entirely in native Rust. While frameworks like TensorFlow or PyTorch abstract away the underlying mathematics to let developers focus strictly on higher-level architecture, using Rust introduces low-level control over memory management, performance optimization, absolute type safety, and strict ownership validation via the borrow checker.

2 Why Tokenization Matters

The boundaries of tokenization dictate what an LLM can ultimately achieve. Consider the following tokenization behavior over fragmented strings of the word “*hello*”:

- ["he", "llo"] → mapped to Token IDs: 530 and 840 respectively.
- [" he", "llo "] → mapped to Token IDs: 460 and 670 respectively.
- ["ol", "leh"] → mapped to Token IDs: 91 and 133 respectively.

Even though the underlying string represents the same word, the resulting Token IDs are completely disjointed. Because these variation boundaries prevent the neural network from effortlessly reversing or manipulating strings at a character level, word-level or unoptimized subword tokenization remains highly inefficient.

3 The Byte Pair Encoding (BPE) Algorithm

To resolve the limits of word-level tokenization, Feste defaults to a character/byte level where base tokens represent raw byte definitions. An 8-bit binary digit represents exactly 256 unique baseline values. Standard English characters consume 1 byte each, whereas complex items like emojis require up to 4 bytes. Each unique byte receives its own dedicated Token ID.

3.1 BPE Token Assignment

The following table illustrates the baseline mapping of raw character bytes to specialized Token IDs:

Character	Byte Value	Initial Token ID
A	65	530
B	66	870
C	67	754
D	68	214
E	69	125
F	70	859
G	71	381
H	72	350
I	73	328
J	74	242
K	75	854

Character	Byte Value	Initial Token ID
L	76	204
M	77	792
N	78	858
O	79	658
P	80	189
Q	81	704
R	82	532
S	83	132
T	84	130
U	85	195
V	86	323
W	87	338
X	88	617
Y	89	716
Z	90	127

The BPE algorithm works by iteratively merging the most frequent adjacent tokens into entirely new, single tokens.

Example: If the pair "th" repeats frequently across a text corpus, its individual character tokens ('t' and 'h') are systematically combined into a single new subword token, such as 344.

Consequently, common structural words consolidate into their own unique tokens, while rare terms break down into familiar base byte sequences that the tokenizer has observed across other training environments.

- **Smaller Vocabularies:** Increase sequence length and fragmentation, forcing the model to construct phrases from tiny components.
- **Larger Vocabularies:** Preserve meaning per token and allow the network to discover conceptual relationships between sequential words more effectively, provided training is sufficient.

4 Implementation Details

The system begins with 256 unique entries mapping hex-encoded bytes to numerical IDs (0–255). During training, new token values are dynamically appended to the lexicon for every unique merge rule. These rules are maintained in an ordered vector, as execution order dictates application significance.

4.1 Prefix Merge Ordering Example

Before encoding a word like "the", the tokenizer must step through its merge rules sequentially. For instance, it applies merge rule 1 (M_1) first to generate token 256 for the pair "th" ($\langle 74 \rangle \langle 68 \rangle$), before eventually applying merge rule 50 (M_{50}) which fuses token 256 with byte $\langle 65 \rangle$ ('e') to generate the complete token 300.

This training pipeline is fully deterministic: it counts pairs, isolates the highest frequency pair, updates the corpus, and repeats without stochastic randomness.

5 Training Performance & Parallelization

The computational bottleneck of BPE lies in counting adjacent token pairs across the training corpus. Instead of a single-threaded architecture, Feste implements the **Rayon** data-parallelism library in Rust to shard text data across multiple execution threads simultaneously.

5.1 Parallel Chunk Boundary Constraints

To prevent missing token merges that cross the split points between independent threads, an explicit **Boundary Check** is performed:

$$\text{Thread 1} \rightarrow [["t", "h"] \quad ["e", "a", "d"]] \iff \text{Thread 2} \rightarrow [["u", "s"] \quad ["e", "d", "t"]]$$

Each thread checks its chunk boundary against the starting token of the adjacent chunk. This ensures spanning pairs (such as a split "th", "the", "us", or "used") are properly preserved and counted. Parallelization is purposefully restricted to pair-counting; executing the merges and updating the internal corpus is kept sequential to prioritize code legibility over over-engineered complexity.

For this educational model, chunks are processed independently; the loss in compression efficiency remains negligible, and BPE enforces strict rules against invalid boundary fracturing.

6 Encoding and Decoding Walkthrough

Let us trace an explicit encoding run for the sequence "To be" based on the following hexadecimal character map:

Table 2: Hexadecimal Character Values

Character	Hex Value
T	54
o	6f
Space	20
b	62
e	65

The baseline byte list is initialized as: $[\langle 54 \rangle, \langle 6f \rangle, \langle 20 \rangle, \langle 62 \rangle, \langle 65 \rangle]$. Assume the tokenizer has learned three ordered merge rules:

- M_1 : $\langle 6f \rangle \langle 20 \rangle \rightarrow \text{"o "}$ $\rightarrow$ Token 256
- M_2 : $\langle 54 \rangle \langle 6f \rangle \rightarrow \text{"To"}$ $\rightarrow$ Token 257
- M_3 : Token 256 + $\langle 62 \rangle \rightarrow \text{"o b"}$ $\rightarrow$ Token 258

The tokenizer processes these constraints sequentially:

1. Applying M_1 transforms the original sequence into $[\langle 54 \rangle, 256, \langle 62 \rangle, \langle 65 \rangle]$.
2. Applying M_2 attempts to find $\langle 54 \rangle \langle 6f \rangle$. Because the byte $\langle 6f \rangle$ was already consumed to create token 256 during the M_1 pass, the rule fails to match and the list remains unchanged.
3. Applying M_3 scans for token 256 followed immediately by byte $\langle 62 \rangle$. This matches perfectly, compressing the collection into its final encoded array: $[\langle 54 \rangle, 258, \langle 65 \rangle]$.

To reverse this process and decode the stream, tokens are mapped back to their raw hexadecimal values as shown below:

Table 3: Decoding Token Reverse Mapping

Token ID	Map Transformation	Hex Byte Code	Expected Output Text
$\langle 54 \rangle$	$\rightarrow$	$\langle 54 \rangle$	T
258	$\rightarrow$	$\langle 6f \rangle \langle 20 \rangle \langle 62 \rangle$	o b
$\langle 65 \rangle$	$\rightarrow$	$\langle 65 \rangle$	e

The resulting hex byte chunks are concatenated together to form $\langle 54 \rangle \langle 6f \rangle \langle 20 \rangle \langle 62 \rangle \langle 65 \rangle$, which is parsed back into the human-readable string "To be" via standard UTF-8 parsing compliance.

7 Training Results & Scaling Metrics

Feste is evaluated across five distinct vocabulary sizes to understand performance changes. Production models (such as GPT-2 or GPT-3) scale much higher, maintaining a vocabulary threshold of 50,257 tokens.

The performance of a tokenizer is measured by its **Compression Ratio (CR)**, which is defined as:

$$CR = \frac{\text{Total Raw Bytes of Input Text}}{\text{Total Encoded Tokens Used}} \quad (1)$$

A higher compression ratio reduces the sequence length passed into the transformer network, lowering active memory consumption and speeding up training loops.

7.1 The Token Embedding Matrix

Once tokens are assigned their integer IDs, they are mapped to high-dimensional vectors. These floating-point numbers are the actual parameters (weights) the model learns during training to understand relationships between tokens. For a baseline vocabulary size of 256 where each token is allocated 768 embedding features, the total embedded parameter space equals:

$$256 \times 768 = 196,608 \text{ parameters}$$

Table 4 shows how Token IDs map to continuous floating-point spaces:

Table 4: Token Embedding Matrix (Parameter Lookup Table Layout)

Token ID	Token String	d_0	d_1	d_2	d_3	...	d_{511}
256	th	0.1245	-0.4512	0.8821	-0.0192	...	-0.0034
257	To	-0.9921	0.0154	0.3409	0.1125	...	0.7710
258	o b	0.3310	-0.2100	-0.5541	0.9982	...	0.1093
300	the	0.5501	0.2219	-0.1123	-0.7761	...	-0.4428
...	...	...	...	...	...	...	...
50256	< endoftext >	-0.1102	0.8842	0.0012	-0.3341	...	0.0001

7.2 Vocabulary Size Cross-Comparisons

Tables 5 and 6 present the empirical data regarding training execution times, sequence compression behaviors, and vocabulary merge breakdowns:

Table 5: Tokenizer Metrics Across Varying Vocabulary Caps

Vocabulary Size	Encoded Length (tokens)	Compression Ratio	Training Time (s)	Encoding Time
256	5,422,721	1.00x	0.00	0.14
512	2,772,080	1.96x	25.54	2.04
1,024	2,189,778	2.48x	77.45	4.88
1,536	1,952,470	2.78x	142.51	7.46
20,534	1,481,106	3.66x	286.34	242.50

Table 6: BPE Merges and Lexicon Structural Breakdown

Vocabulary Size	Base Vocabulary	Additional Merges	Total Merges Breakdown
256	256	0	256 (Base) + 0 merges
512	256	256	256 (Base) + 256 merges
1024	256	768	256 (Base) + 768 merges
1536	256	1,280	256 (Base) + 1,280 merges
20,534	256	20,278	256 (Base) + 20,278 merges

7.3 Trade-offs & Diminishing Returns

While expanding the vocabulary size increases the compression ratio, the gains scale with diminishing returns. This occurs because the initial iterations capture highly frequent, repeating character patterns. Once these core patterns are merged, subsequent rules target rare phrases or outliers that do not repeat often enough to significantly boost compression. Additionally, without advanced sampling optimizations, training time scales linearly with vocabulary size.

7.4 The Data Scarcity Problem

If an extensive vocabulary size (such as 1536 or higher) is forced onto a relatively small training dataset, the tokenizer is forced to remember small phrases or rare word structures that appear only once. This leads to **embedding collapse**, causing the language model to interpret rare sequences as random statistical noise. Conversely, a very small vocabulary causes long sequence fragmentations, which slows down context processing. Selecting a vocabulary size proportionate to the dataset volume is vital.

8 Strategic Architectural Choices

The overarching design goal of Feste is to choose structural clarity over extreme performance optimization. Understanding how tokenization works step-by-step is prioritized over absolute execution speed.

8.1 String-Based Token Representation Trade-offs

Production tokenizers save memory by keeping merges mapped strictly as paired integers: `(token_a, token_b) = new_token`. Feste, by contrast, deliberately retains hexadecimal character text representations (`<66>`, `<6f>`) within its active token configurations.

Table 7: Pros and Cons of String-Based Token Layouts

Pros	Cons
• Highly debuggable and human-readable. • Explicitly clear; directly exposes what each token captures.	• Wastes active system memory. • Demands high storage and continuous parsing overhead.

8.2 Vocabulary Management

For plain English text, mapping token sequences to UTF-8 standards ensures perfect, error-free encoding and decoding cycles. However, this strategy is less optimized for non-English languages, which often require multiple bytes per character. Production-grade tokenizers (like `tiktoken_rs`) address these multi-byte issues by pruning low-frequency merges. Feste, by contrast, purposefully retains all merge steps—including outliers—to ensure the entire pipeline can be audited.

9 Part 2: Tensor Operations

The next phase shifts focus to constructing a custom tensor operations calculation library in native Rust to power Feste’s transformer layers.

Tensor computations form the mathematical core of the transformer model. Operations ranging from multi-head self-attention mechanisms to layer normalization simplify down to fundamental matrix arithmetic performed on multi-dimensional numerical arrays. Building this engine from scratch reveals how abstract transformer equations translate into concrete, parallelized calculations on physical computer hardware.